

CONFIDENTIAL



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
UTM Johor Bahru

UNIVERSITI TEKNOLOGI MALAYSIA
FINAL EXAM SEMESTER II, 2023/2024

SUBJECT CODE : SECJ2154

SUBJECT NAME : OBJECT ORIENTED

PROGRAMMING YEAR/PROGRAM : 2 (SECB/ SECJ/ SECP/

SECR/ SECV) DURATION : 3 HOURS

DATE :

VENUE :

INSTRUCTIONS:

THIS EXAMINATION BOOK CONSISTS OF 2 SECTIONS:

SECTION A: Structured Questions 70 Marks

SECTION B: Programming 30 Marks

ANSWER ALL QUESTIONS IN THE ANSWER BOOKLET.
THE QUESTION BOOKLET MUST BE RETURNED WITH THE ANSWER BOOKLET

(Please write your lecturer name and section in your answer booklet)

Name	MUHAMMAD AFIQ DANIAL BIN ROZAIDIE
IC	041204011037
Year/Program	2/SECPH
Section	02
Lecturer Name	DR ZURAINI

This question paper consists of **12 pages** excluding this page.

PART A: STRUCTURED QUESTION**[70 MARKS]**

Part A consists of five (5) structured questions. Write your answers in the answer booklet.

Question 1 (10 marks)

Analyze the following Java program in Figure 1.1 below :

```
1  import java.util.ArrayList;
2  import java.util.Vector;
3
4  public class AQList{
5      public static void main(String[] args) {
6          ArrayList<String> arrayList = new ArrayList<>();
7          arrayList.add("Banana");
8          arrayList.add("Olive");
9          arrayList.add("Date");
10         arrayList.add("Fig");
11         System.out.println("List: " + arrayList);
12
13         Vector<String> vector = new Vector<>();
14         vector.add("Ants");
15         vector.add("Bee");
16         vector.add("Cow");
17         vector.add("Dog");
18         System.out.println("Vector: " + vector);
19
20         arrayList.set(1, "Grape");
21         arrayList.remove(0);
22
23         vector.remove("Cow");
24         vector.setElementAt("Fish", 2);
25
26         System.out.println("List: " + arrayList);
27         System.out.println("Vector: " + vector);
28     }
29 }
```

Figure 1.1: AQList class definitions

Answer the given question based on Figure 1.1 AQList class

- i) The output for line 11 is : List: [Banana, Olive, Date, Fig] (2 marks)
- ii) The output for line 18 is : Vector: [Ants, Bee, Cow, Dog] (2 marks)
- iii) The output for line 26 is : List: [Grape, , Date, Fig] (3 marks)
- iv) The output for line 27 is : Vector: [Ants, Bee, Fish] (3 marks)

Question 2 (15 marks)

Analyze the class diagram and incomplete program that represents the class `House`, `Furniture`, and `Room` relationship.

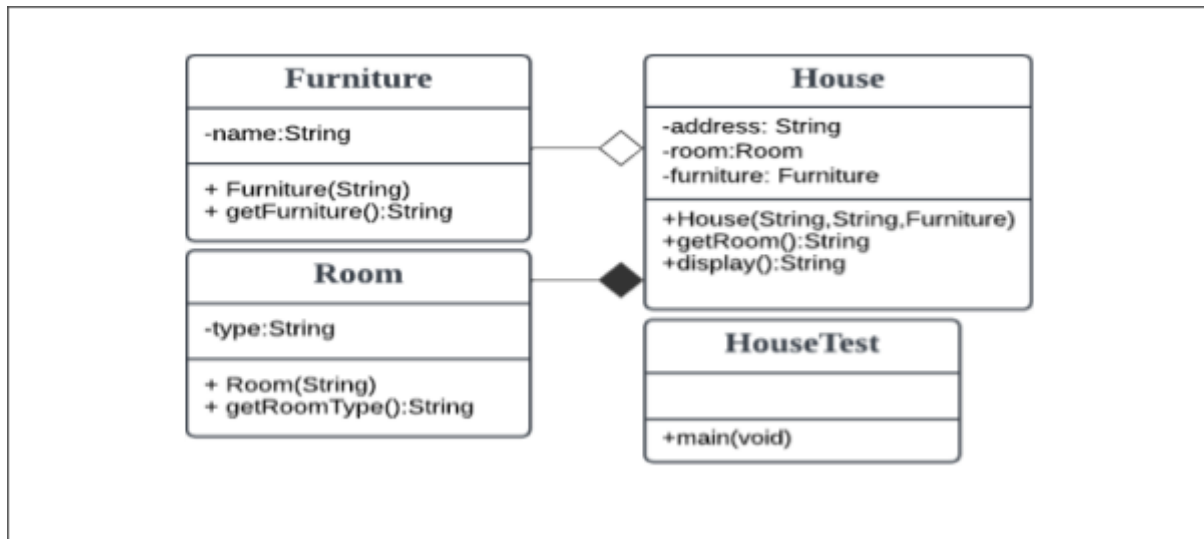


Figure 2.1: Class Diagram of `House`, `Furniture`, `Room`

Given Figure 2.2, class `HouseTest` for the main method implementation.

```
1 // class HouseTest
2 public class HouseTest
3 {
4     public static void main(String[] args) {
5         Furniture furl = new Furniture("Sofa");
6         House house = new House("No 2 Taman U", "Bedroom", furl);
7         System.out.println(house.display());
8     }
9 }
```

Figure 2.2: `HouseTest` class definitions

```
1 //class House
2 class House {
3     private String address;
4     private Room room;
5     private Furniture furniture;
6
7     public House(String address, Room room, Furniture
8                 furniture) {
9         this.address = address;
10        this.room = room;
11        this.furniture = furniture;
12    }
13
14    public String getRoom(){
15        return room.getRoomType();
16    }
17
18    public String display(){
19        return "Address: " + address + "\nFurniture: " +
20               furniture.getFurniture() + "\nRoom: " + getRoom();
21    }
22 }
```

19	}
20	}

Figure 2.3: **House** class definitions

```

1 //class Room
2 class Room {
3     private String type;
4
5     public Room(String type) {
6         this.type = type;
7     }
8
9     public String getRoomType() {
10        return type;
11    }
12 }

```

Figure 2.4: Room class definitions

```

1 // class Furniture
2 class Furniture {
3     private String name;
4
5     public Furniture(String name) {
6         this.name = name;
7     }
8
9     public String getFurniture() {
10        return name;
11    }
12 }

```

Figure 2.5: Furniture class definitions

```

Address: No 2 Taman U
Furniture: Sofa
Room: Bedroom

```

Figure 2.6: Output

Construct the correct codes according to the given class diagram (Figure 2.1). The code should generate the specified output as in Figure 2.6.

- i) Figure 2.3 at line 3-line 5: define the attributes of the `House` class (3 marks)
- ii) Figure 2.3 at line 7-line 10: complete code of `House` class constructor. (6 marks)
- iii) Figure 2.3 at line 14: complete code of `getRoom()` method that return a value of room type (1 mark)
- iv) Figure 2.3 at line 18: complete code of `display()` method that displays the output as shown in Figure 2.6. (3 marks)
- v) Figure 2.4 at line 10: complete code of `getRoomType()` (1 mark)

vi) Figure 2.5 at line 10, complete code of `getFurniture()` .

(1 mark)

Question 3 (15 marks)

3.1 Examine the following Java source code in Figure 3.1.

```
1 class Item {
2     private String name;
3
4     public Item(String name) {
5         // i) assign Item class attribute's values
6         this.name = name;
7     }
8
9     public String getName() {
10        return name;
11    }
12 }
13
14 class Product extends Item {
15     private double price;
16     private int quantity;
17
18     public Product(String name, double price, int quantity) {
19         // ii) assign subclass and superclass attribute values
20         super(name)
21
22         this.price = price; this.quantity = quantity;
23     }
24
25     public Product(String name, double price) {
26         iii) assign subclass and superclass attribute values
27
28         super(name);
29
30         this.price = price;
31     }
32
33     public void printInfo() {
34         iv) Print name, quantity, and price
35         System.out.println(name);
36         System.out.println(quantity);
37         System.out.println(price);
38     }
39
40     public class TestProduct {
41         public static void main(String[] args) {
42             Product product1 = new Product("Book", 15.99, 10);
43             Product product2 = new Product("Pen", 1.49);
44             product1.printInfo();
45             product2.printInfo();
46         }
47     }
```

Figure 3.1: **Product**, **Item**, and **TestProduct** class definitions

Answer the following questions based on the classes defined in Figure 3.1 above.

- i) Assign values for `Item` superclass attributes (1 mark)
- ii) Assign values to subclass attributes and its superclass inherited attribute (3 mark)
- iii) Assign values to subclass attributes and its superclass inherited attribute (2 marks)
- iv) Print the `name`, `price`, and `quantity` (3 marks)

3.2 Analyze the following Java source code in Figure 3.2.

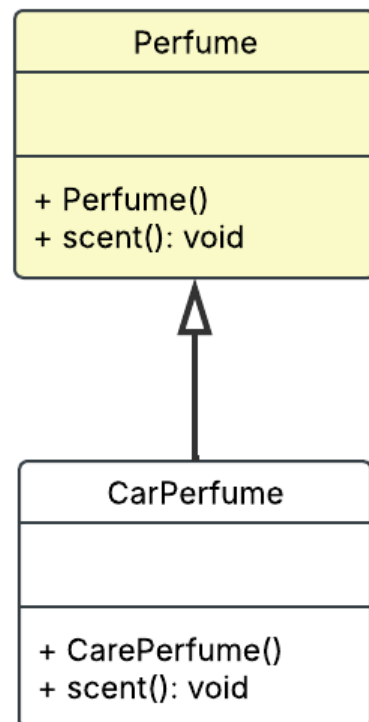
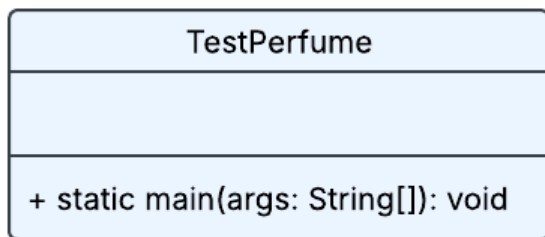
```
1 class Perfume {
2
3     public Perfume() {
4         System.out.println("Super-constructor");
5     }
6
7     public void scent() {
8         System.out.println("Pleasant scent.");
9     }
10 }
11
12 class CarPerfume extends Perfume {
13
14     public CarPerfume() {
15         System.out.println("Sub-constructor");
16     }
17
18     public void scent() {
19         System.out.println("Refreshing scent.");
20     }
21 }
22
23 public class TestPerfume {
24     public static void main(String[] args) {
25         CarPerfume myCarPerfume = new CarPerfume();
26         myCarPerfume.scent();
27     }
28 }
```

Figure 3.2: `Perfume`, `CarPerfume`, and `TestPerfume` class definitions

Based on the Java source code in Figure 3.2 above, complete the following tasks (i – iii):

- (i) Draw the UML class diagram that represents of Java source code in Figure 3.2. (2 marks)
- (ii) What is the output of the Java source code in Figure 3.2? (3 marks)
- (iii) What is the significance of the `scent()` method in both the `Perfume` and `CarPerfume` classes. (1 mark)

i)



ii)

Super-constructor

Sub-constructor

Refreshing scent

iii)

Method overriding

Question 4 (15 Marks)

4.1 Examine the program in Figure 4.1. Write the correct output of this program. (4 marks)

```
1 class One {
2     protected int x =
3     4;
4
5     void setX(int a){
6         x=a;
7     }
8     protected int
9     getX(){ return x;}
10 }
11
12 class Two extends One {
13     protected int x = 8;
14
15     protected int getX(){
16         return x; }
17
18     public int getB(){
19         return x;
20     }
21 }
22
23 public class Test{
24     public static void main(String [] args){
25         One a = new One();
26         One b = new Two();
27         System.out.println(a.getX());
28         System.out.println(b.getX());
29         System.out.println(a.x);
30         System.out.println(b.x);
31     }
32 }
```

Figure 4.1: One, Two, and Test class definitions

Output:

4
8
4
4

4.2 Examine the following Java program that demonstrates the interface and abstract class through the definition of **Salary**, **Employee**, **Manager**, **Programmer** and **TestSalary** classes. Fill in the blanks with the correct code to complete the program. The expected output shown in figure 4.2. (11 marks)

```

1 interface Salary {
2     double calculateSalary();
3 }
4 abstract class Employee implements Salary {
5     protected String name;
6     _____String role;
7
8     public Employee(String name, String role) {
9         this.name = name;
10        this.role = role;
11    }
12
13    public _____double calculateSalary();
14
15    public void printDetails() {
16        System.out.println("Role: " + role);
17        System.out.println("Name: " + name);
18        System.out.println("Salary: RM " + _____);
19    }
20 }
21
22 class Manager extends _____{
23     public Manager(String name) {
24         _____(name, "Manager");
25     }
26
27     public double _____{
28
29         return 9000.0;
30     }
31 }
32
33 class Programmer extends Employee {
34     public Programmer(String name) {
35         _____(name, "Programmer");
36     }
37
38     public double _____{
39
40         return 5600.0;
41     }
42 }
43
44 public class TestSalary {
45     public static void main(String[] args) {
46         Employee manager = new _____("Faiz Ali");
47         Employee programmer = new _____("Ely Tan");
48         _____printDetails();
49
50         System.out.println();
51
52         _____printDetails();
53     }
54 }
55
56 }

```

Manager: Faiz Ali
Role: Manager
Salary: RM 9000.0

Programmer: Ely Tan
Role: Programmer
Salary: RM 5600.0

Figure 4.2: Expected output

```
abstract class Employee implements Salary {
    protected String name;
    protected String role;

    public abstract double calculateSalary();

    public void printDetails() {
        System.out.println("Role: " + role);
        System.out.println("Name: " + name);
        System.out.println("Salary: RM " + calculateSalary());
    }
}

class Manager extends Employee {
    public Manager(String name) {
        super(name, "Manager");
    }

    public double calculateSalary() {
        return 9000.0;
    }
}

class Programmer extends Employee {
    public Programmer(String name) {
        super(name, "Programmer");
    }

    public double calculateSalary() {
        return 5600.0;
    }
}

public class TestSalary {
    public static void main(String[] args) {
        Employee manager = new Manager ("Faiz Ali");
        Employee programmer = new Programmer ("Ely Tan");

        manager.printDetails();
        System.out.println();
        programmer.printDetails();
    }
}
```

Question 5 (15 Marks)

5.1 Given the class `CalculatorDemo` in Figure 5.1 below:

```
1 public class CalculatorDemo {
2     public static void main(String[] args) {
3         boolean flag = true;
4         if (flag = false) { //
5             System.out.println("This is a syntax error.");
6         }
7
8         int i = 0;
9         int j = 10;
10        int x = calculate(i,j);
11        int y = j/i;
12        int[] arr = new int[5];
13        System.out.println(arr[10]);
14
15    }
16
17    public int calculate (int a, int b){
18        int result;
19        result = b + a;
20
21    }
22 }
```

Figure 5.1: Java program of class `CalculatorDemo`

Trace the provided Java program and identify the line number and type of error (syntax error, runtime error, or logical error) at each problematic point in the program.

Line Number	Type of error
4	Syntax error
11	Runtime error
13	Runtime error
20	Syntax error

(8 marks)

5.2 Given the class `ExceptionExample` in Figure 5.2 demonstrating exception handling for array creation, input validation, and arithmetic operations using try-catch blocks. Complete the following Java program by filling in the blanks:

(7 marks)

```

1 import java.util.Scanner;
2
3 public class ExceptionExample {
4     public static void main(String[] args) {
5         Scanner scanner = new Scanner(System.in);
6
7         try {
8             // Part 1: Creating an array with user-defined size
9             System.out.println("Enter the size of the array:");
10            int size = scanner.nextInt();
11            if (size <= 0) {
12                throw new IllegalArgumentException("Array size must be a positive
13                    integer."); // 1.5 mark
14            }
15
16            int[] array = new int[size];
17
18            for (int i = 0; i <= size; i++) {
19                System.out.println("Enter element " + i + ":");
20                array[i] = scanner.nextInt();
21            }
22
23            // Part 2: Performing division
24            System.out.println("Enter a divisor:");
25            int divisor = scanner.nextInt();
26            if (divisor == 0) {
27                throw new ArithmeticException("Cannot divide by zero."); // 1.5 mark
28            }
29            int result = element / divisor;
30            System.out.println("Result of division: " + result);
31
32            // Part 3:
33        } catch (IllegalArgumentException e) 1.5 mark {
34            System.out.println("Index out of bounds. Please enter a
35                valid index.");
36        } catch (ArithmeticException e) 1.5 mark {
37            System.out.println("Arithmetic error: " +
38                e.getMessage()) 1 mark;
39        } finally {
40            scanner.close();
41        }
42    }
43 }

```

Figure 5.2: ExceptionExample program.

PART B: PROGRAMMING QUESTION

[30 MARKS]

Part B consists of **ONE (1)** programming question. Write your answers in the answer booklet.

Write a complete Java program that represents the UML class diagram in Figure 1 and produces the expected output depicted in Figure 2. Implement all the classes (excluding the **Order** and **FoodOrderingSystem** classes) with the specified instance variables (attributes) and methods as outlined in the diagram. The purpose of each method is as the name implies, and some of them are further explained below. Note that the definitions of the **Order** and **FoodOrderingSystem** classes are fully given in Figure 3.

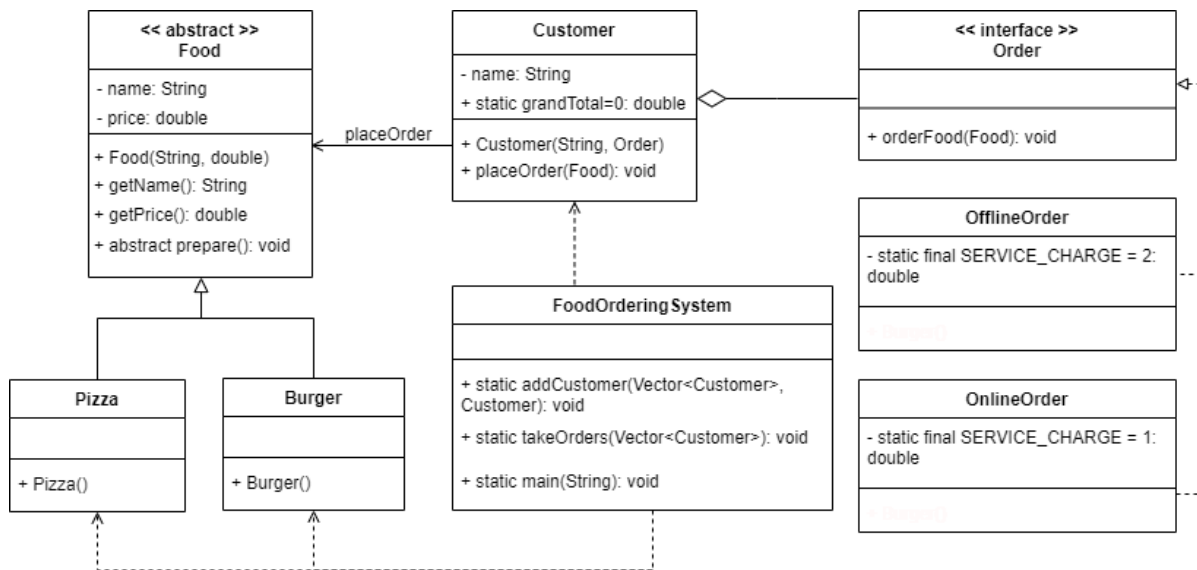


Figure 1: UML Class Diagram

```
Preparing Hawaiian Chicken...
Hawaiian Chicken ordered online.
Total: RM41.90

Preparing Double Cheeseburger...
Double Cheeseburger ordered online.
Total: RM19.35

Preparing Hawaiian Chicken...
Hawaiian Chicken ordered offline.
Total: RM42.90

Preparing Double Cheeseburger...
Double Cheeseburger ordered
offline. Total: RM20.35

Grand Total Sales = RM124.50
```

Figure 2: Expected output of the program

You are required to do the following tasks:

- a) Define a class named **OnlineOrder**: (5 marks)
- This class should implement the **Order** interface.
 - Declare a constant (`SERVICE_CHARGE`) representing the service charge for online orders.
 - Define an `orderFood()` method that prepares the food, calculates the cost for online order food by adding the price of food with the service charge, which is RM1.00 for online orders and updates the grand total sales. Then, display the order and the cost of the order on the screen (refer to Figure 2).
- b) Define a class named **OfflineOrder**: (5 marks)
- This class should implement the **Order** interface.
 - Declare a constant (`SERVICE_CHARGE`) representing the service charge for offline orders.
 - Define an `orderFood()` method that prepares the food, calculates the cost for offline order food by adding the price of food with the service charge, which is RM2.00 for offline orders and updates the grand total sales. Then, display the order and the cost of the order on the screen (refer to Figure 2).
- c) Define an abstract class named **Food**: (6 marks)
- Define a constructor that takes arguments and initializes the member attributes with passed arguments.
 - Define `getName()` method that returns the food name.
 - Define `getPrice()` method that returns the food price.
 - Declare an abstract method named `prepare()`. Subclasses must define this method.
- d) Define a class named **Pizza**: (4 marks)
- This class should inherit from the **Food** class.
 - Define a constructor and initializes the superclass's attributes with "Hawaiian Chicken" as name and RM40.90 as price.
 - Define `prepare()` method that displays pizza preparing message.
- e) Define a class named **Burger**: (4 marks)
- This class should inherit from the **Food** class.
 - Define a constructor and initializes the superclass's attributes with "Double Cheeseburger" as name and RM18.35 as price.
 - Define `prepare()` method that displays burger preparing message.

f) Define a class named **Customer**:

(6 marks)

- Define a constructor that takes arguments and initializes the member attributes with passed arguments.
- Define `placeOrder()` method that places the food order made by the customer.

```
1 // FoodOrderingSystem.java
2 import java.util.*;
3
4 interface Order {
5     void orderFood(Food food);
6 }
7
8 class FoodOrderingSystem {
9     //Method to add a customer to the system
10    public static void addCustomer(Vector<Customer> cList, Customer c) {
11        cList.add(c);
12    }
13
14    // Method to take orders from all customers in the system
15    public static void takeOrders(Vector<Customer> cList) {
16        for (Customer c: cList) {
17            c.placeOrder(new Pizza());
18            c.placeOrder(new Burger());
19        }
20    }
21
22    public static void main(String[] args) {
23        // Create a new customer list
24        Vector<Customer> cList = new Vector<>();
25
26        //Add customers to the system
27        addCustomer(cList, new Customer("Haris", new OnlineOrder()));
28        addCustomer(cList, new Customer("Aliya", new OfflineOrder()));
29
30        //Take orders from all customers
31        takeOrders(cList);
32        System.out.printf("Grand Total Sales = RM%.2f\n",
33            Customer.grandTotal);
34    }
35 }
```

Figure 3: Order and FoodOrderingSystem class definitions

```

// FoodOrderingSystem.java
import java.util.*;

interface Order {
    void orderFood(Food food);
    double getCharge();
}

class OnlineOrder implements Order {
    private static final double SERVICE_CHARGE = 1;

    public void orderFood(Food food){
        System.out.println(food.getName() + " ordered online.");
        System.out.printf("Total: RM%.2f\n", (food.getPrice()+SERVICE_CHARGE));
        System.out.println();
    }
    public double getCharge(){
        return SERVICE_CHARGE;
    }
}

class OfflineOrder implements Order {
    private static final double SERVICE_CHARGE = 2;

    public void orderFood(Food food){
        System.out.println(food.getName() + " ordered offline.");
        System.out.printf("Total: RM%.2f\n", (food.getPrice()+SERVICE_CHARGE));
        System.out.println();
    }
    public double getCharge(){
        return SERVICE_CHARGE;
    }
}

abstract class Food {
    private String name;
    private double price;

    public Food(String name, double price) {
        this.name = name;
        this.price = price;
    }
    public String getName() {
        return name;
    }
    public double getPrice() {
        return price;
    }
    abstract void prepare();
}

```

```

class Pizza extends Food {
    public Pizza() {
        super("Hawaiian Chicken", 40.90);
    }
    @Override
    void prepare() {
        System.out.println("Preparing " + getName() + "...");
    }
}

class Burger extends Food {
    public Burger() {
        super("Double Cheeseburger", 18.35);
    }
    @Override
    void prepare() {
        System.out.println("Preparing " + getName() + "...");
    }
}

class Customer {
    private String name;
    public static double grandTotal=0;
    private Order order;

    public Customer(String name, Order order){
        this.name = name;
        this.order = order;
    }
    public void placeOrder(Food food){
        food.prepare();
        order.orderFood(food);
        grandTotal += food.getPrice() + order.getCharge();
    }
}

public class FoodOrderingSystem {
    // Method to add a customer to the system
    public static void addCustomer(Vector<Customer> cList, Customer c) {
        cList.add(c);
    }

    // Method to take orders from all customers in the system
    public static void takeOrders(Vector<Customer> cList) {
        for (Customer c : cList) {
            c.placeOrder(new Pizza());
            c.placeOrder(new Burger());
        }
    }
}

```

```
public static void main(String[] args) {  
    // Create a new customer list  
    Vector<Customer> cList = new Vector<>();  
    // Add customers to the system  
    addCustomer(cList, new Customer("Haris", new OnlineOrder()));  
    addCustomer(cList, new Customer("Aliya", new OfflineOrder()));  
    // Take orders from all customers  
    takeOrders(cList);  
    System.out.printf("Grand Total Sales = RM%.2f\n",  
        Customer.grandTotal);  
}  
}
```